Kubernetes Homelab: Calico + firewalld

Step-by-step troubleshooting, permanent configuration, and firewall command reference
Documented from the October 2026 troubleshooting session. Covers Rocky Linux 9, Calico IP-in-IP, CoreDNS, Argo CD and Jenkins.

1. Your cluster and what failed

Node	LAN address	Role
master	192.168.178.49	Control plane; CoreDNS Pods observed here
node01	192.168.178.51	Worker
node02	192.168.178.52	Worker; DNS test Pod and NodePort access

Observed networking: Kubernetes v1.28.15; Calico Pod CIDR 172.16.0.0/16; Calico IP-in-IP Always; VXLAN Never; NAT outgoing true. kube-dns Service IP 10.96.0.10. Jenkins NodePort 30080; Argo CD HTTPS NodePort 30081.

Initial symptom: Argo CD could not resolve github.com. A Pod on node02 could reach 1.1.1.1 and resolve github.com using 1.1.1.1, but could not reach the CoreDNS Pod IPs 172.16.219.72 / .73 or the kube-dns ClusterIP 10.96.0.10. Error: No route to host.

2. How we isolated the problem - one test at a time

Test A: Confirm Pod internet routing

```
kubectl exec -n argocd network-test -- ping -c 3 -W 2 1.1.1.1
kubectl exec -n argocd network-test -- nslookup github.com 1.1.1.1
```

Observed: both passed. This ruled out a total loss of outbound Pod connectivity. It did not prove internal Pod routing was healthy.

Test B: Check Kubernetes DNS and direct CoreDNS reachability

```
kubectl exec -n argocd network-test -- \
  nslookup kubernetes.default.svc.cluster.local 10.96.0.10
kubectl exec -n argocd network-test -- nslookup github.com 10.96.0.10
kubectl exec -n argocd network-test -- nslookup github.com 172.16.219.72
```

Observed initially: failed with No route to host. The CoreDNS IPs were valid at the time; always re-check Pod IPs before repeating this test.

Test C: Inspect Calico encapsulation

```
kubectl get ippools.crd.projectcalico.org \
  -o custom-columns=NAME:.metadata.name,CIDR:.spec.cidr,\
IPIP:.spec.ipipMode,VXLAN:.spec.vxlanMode,NAT:.spec.natOutgoing
```

Observed: default-ipv4-ippool; 172.16.0.0/16; IPIP Always; VXLAN Never; NAT true. IP-in-IP uses IP protocol 4, not TCP/UDP port 4.

Test D: Identify the firewall as a factor

```
sudo systemctl is-active firewalld
sudo firewall-cmd --list-all
```

firewalld was active. Allowing IP protocol 4 alone on master and node02 did not restore DNS. When firewalld was stopped on master for a short diagnostic test, both direct CoreDNS Pod DNS and kube-dns ClusterIP DNS worked. firewalld was then restarted.

Test E: Trust Calico interfaces temporarily

```
sudo firewall-cmd --zone=trusted --add-interface=tunl0
sudo firewall-cmd --zone=trusted --add-interface=calic54cb46bd73
sudo firewall-cmd --zone=trusted --add-interface=calicde4b82c7ef
sudo firewall-cmd --zone=trusted --add-interface=calia29175e05aa
```

Observed: DNS worked with firewalld running. These particular cali* names were from master at the time and can change as Pods are recreated. Do not copy these exact names to other nodes.

3. Permanent firewalld configuration - documented tested state

The following settings were made permanent on master, and DNS from the test Pod on node02 was verified after a firewalld reload. The configuration was also proposed for the two workers. This establishes a successful reload test, not yet a confirmed reboot test on every node.

Run on each node, one at a time (master, node01, node02)

```
sudo systemctl enable --now firewalld && \
sudo firewall-cmd --permanent --zone=trusted --add-interface=tunl0 && \
sudo firewall-cmd --permanent --zone=trusted --add-source=172.16.0.0/16 && \
sudo firewall-cmd --permanent --zone=public \
  --add-rich-rule='rule family="ipv4" source address="192.168.178.0/24" protocol
\value="4" accept' && \
sudo firewall-cmd --check-config && \
sudo firewall-cmd --reload && \
sudo firewall-cmd --get-active-zones
```

Meaning: start/enable firewalld; persistently classify tunl0 as trusted; classify traffic sourced from the Pod CIDR as trusted; permit IP-in-IP packets from the trusted LAN; validate configuration; reload; inspect active zones.

Security warning: trusted zone ACCEPT is broad, especially for 172.16.0.0/16. Use only when that CIDR is dedicated to Pods and your homelab network is trusted. The 192.168.178.0/24 source is wider than the three Kubernetes nodes. A tighter production configuration would limit peers and forwarding more precisely. Avoid adding unconditional ACCEPT rules for cali+ unless separately justified.

ALREADY_ENABLED warnings mean a setting was already present; they are not failures. The successful master output showed public on ens18/ens19 and trusted on tunl0 plus source 172.16.0.0/16.

4. Validate DNS with a fresh test Pod

The original network-test Pod eventually reached phase Succeeded. kubectl exec does not work against a completed Pod. Recreate it as follows from master:

```
kubectl delete pod network-test -n argocd --ignore-not-found
kubectl run network-test -n argocd --image=busybox:1.36 \
  --restart=Never \
  --overrides='{"spec":{"nodeName":"node02"}}' \
  --command -- sleep 86400
kubectl wait -n argocd --for=condition=Ready \
  pod/network-test --timeout=120s

kubectl exec -n argocd network-test -- nslookup github.com 10.96.0.10
kubectl exec -n argocd network-test -- \
  nslookup kubernetes.default.svc.cluster.local 10.96.0.10
```

Observed after master permanent reload: github.com resolved to 140.82.121.4 and kubernetes.default.svc.cluster.local resolved to 10.96.0.1. Both tests passed. The public GitHub address may

change.

To verify node01 independently, create a separate test Pod pinned to node01 and repeat both DNS tests. Test again after applying each worker firewall configuration, then after any reboot.

5. Verify Jenkins and Argo CD NodePorts

```
kubectl get svc -n jenkins
kubectl get svc -n argocd argocd-server
kubectl get pods -n jenkins -o wide
kubectl get pods -n argocd -o wide
```

Jenkins: http://192.168.178.52:30080/login
Argo CD: https://192.168.178.52:30081

```
curl -v --connect-timeout 5 http://192.168.178.52:30080
curl -vk --connect-timeout 5 https://192.168.178.52:30081
```

Observed on node02: Jenkins returned HTTP 403 because the anonymous user lacked Hudson.Read; this was a functioning authentication response, not a network failure. Argo CD returned HTTP 200 with a self-signed TLS certificate. MacBook curl reached Jenkins, and both apps opened in Safari. Chrome displayed ERR_ADDRESS_UNREACHABLE; that is a separate browser-specific issue, not evidence that the Kubernetes Services were down.

6. Firewall command reference - every command used or discussed

Command	Purpose / notes
systemctl is-active firewalld	Shows active/inactive status.
systemctl enable --now firewalld	Starts firewalld now and enables startup on reboot.
systemctl stop firewalld	Temporary diagnostic only; reduces firewall protection.
systemctl start firewalld	Restores firewalld after diagnostic stop.
firewall-cmd --list-all	Shows current/default active zone settings.
firewall-cmd --get-active-zones	Shows zones with interfaces or sources.
firewall-cmd --zone=public --list-all	Shows public zone rules, including NodePort range.
firewall-cmd --zone=trusted --list-all	Shows trusted interfaces and source CIDRs.
firewall-cmd --list-all-policies	Shows firewalld policy objects.
firewall-cmd --zone=trusted --add-interface=tunl0	Runtime-only trust for Calico IPIP tunnel.
firewall-cmd --zone=trusted --add-interface=<cali-name>	Runtime-only trust for a specific Pod veth; dynamic name.
firewall-cmd --permanent --zone=trusted --add-interface=tunl0	Persists tunnel classification.
firewall-cmd --permanent --zone=trusted --add-source=172.16.0.0/16	Persists broad Pod-CIDR trust.
firewall-cmd --add-rich-rule=... protocol value="4" accept	Runtime IP-in-IP allow rule (source LAN).
firewall-cmd --permanent --zone=public --add-rich-rule=...	Persistent IP-in-IP allow rule.
firewall-cmd --check-config	Checks permanent configuration syntax.
firewall-cmd --reload	Applies permanent rules; runtime-only rules disappear.
firewall-cmd --list-rich-rules	Lists rich rules for default zone.
iptables -S FORWARD	Shows legacy/compat forwarding chain syntax.
iptables -L FORWARD -n -v	Shows forwarding counters and chains.
nft list ruleset	Inspect nftables rules; firewalld may use nftables.

Other diagnostic commands

```
ip -br link | grep -E "cali|tunl|ens"
ip route get 172.16.219.72
ip route
ip -brief address
kubectl get pods -n kube-system -o wide | grep -E "calico|coredns|kube-proxy"
kubectl get endpoints kube-dns -n kube-system -o wide
kubectl get networkpolicy -A
kubectl get globalnetworkpolicy.crd.projectcalico.org
```

The iptables FORWARD output on master showed Calico and kube-proxy chains and a default ACCEPT policy. That did not rule out filtering by firewalld through nftables. The direct-rule experiments suggested earlier (FORWARD -i cali+ / -o cali+) were not reported as executed or verified and are intentionally excluded from the recommended permanent block.

7. Rollback, safety, and next verification

Before changing firewalld, retain Proxmox console access. Change one node at a time. If a new permanent rule causes problems, remove that same rule using --remove-interface, --remove-source, or --remove-rich-rule with the exact original arguments, then run firewall-cmd --reload. Do not blindly flush iptables or nftables; Calico and kube-proxy depend on their rules.

```
sudo firewall-cmd --permanent --zone=trusted --remove-interface=tunl0
sudo firewall-cmd --permanent --zone=trusted --remove-source=172.16.0.0/16
sudo firewall-cmd --permanent --zone=public \
  --remove-rich-rule='rule family="ipv4" source address="192.168.178.0/24" protocol
\value="4" accept'
sudo firewall-cmd --reload
```

Rollback commands are examples to remove only the three documented permanent additions; use them only if you intend to undo this configuration. After worker rollout, validate CoreDNS from Pods on each node, NodePorts from the MacBook, and persistence after a planned reboot.

Clean up: kubectl delete pod network-test -n argocd --ignore-not-found